SAPIN: A Reference Architecture and Validation Harness for Saccade-Aware Robotic Fly Interception

Anonymous Author(s)
Affiliation
Address
email

Abstract

Intercepting an erratically moving target such as a free-flying insect is a challenging visuomotor control problem that demands high-speed perception, predictive trajectory forecasting, and reactive planning within a tight latency budget. We present SAPIN (Saccade-Aware Predictive Interception Network), a reference architecture and validation harness for robotic fly interception. The architecture combines a lightweight CNN-based detector for real-time fly localization, a Transformer-based trajectory predictor with explicit saccade detection and multi-modal (K=3) output, a geometric planner that switches between sweep and precision modes based on detected saccade probability, and a PPO-compatible continuous control policy. We provide a validation harness covering shape correctness, gradient flow, numerical stability (bfloat16-safe), synthetic micro-benchmarks (action diversity, detection stability under noise, trajectory consistency, throughput profiling), and parameter efficiency. The full pipeline comprises approximately 3.4M parameters with a design latency budget of 11 ms per frame targeting 90 Hz inference; real-time performance on target edge hardware has not yet been verified. All components pass comprehensive unit tests, and the harness supports 10 single-field ablation configurations for systematic architecture exploration (one ablation, the GRU predictor swap, is a config-only change pending a full module implementation). We provide the reference implementation and validation harness as accompanying artifacts to support reproducible follow-up work. **No trained-model evaluation on real-world data is presented**; the architecture and harness are intended as a reproducible starting point for subsequent training and empirical studies.

1 Introduction

The task of intercepting a free-flying insect with a robotic manipulator is a visuomotor control problem that combines real-time object detection, trajectory forecasting under multi-modal uncertainty, and reactive planning subject to tight latency constraints. *Drosophila melanogaster*, a common model organism, presents a particularly challenging target: its flight consists of straight cruising segments averaging 13 s punctuated by ballistic saccades—rapid turns of 20–180° completed in approximately 50 ms (7; 8). Once initiated, a saccade is ballistic and cannot be altered mid-turn, creating a narrow window during which the fly's trajectory is both extreme and predictable.

Existing approaches to robotic insect interception fall into two broad categories. Reactive visual servoing controllers ("chase" policies) are simple and computationally cheap but suffer from latency-induced miss rates during rapid turns, as the control delay of the robotic arm causes the end-effector to lag behind the target. High-speed track-and-pursuit systems using dedicated hardware, such as the Fast Lock-On (FLO) system (15), achieve sub-10 ms tracking latency with retroreflective markers, but require specialized sensing infrastructure and are not easily generalised to marker-free settings.

Submitted to 37th Conference on Neural Information Processing Systems (NeurIPS 2023). Do not distribute.

This gap suggests a design opportunity: can a lightweight perception pipeline, combined with explicit saccade detection and multi-modal trajectory prediction, match the interception performance of reactive baselines without requiring dedicated high-speed tracking hardware? The key insight is that saccades, while extreme, are predictable once detected, and that a compliant catching mechanism (a net or padded swatter) relaxes the required spatial precision, making a broad sweep strategy viable during periods of high trajectory uncertainty.

In this work we present SAPIN (Saccade-Aware Predictive Interception Network), a reference architecture designed to test this hypothesis. SAPIN comprises four neural modules—a perception module (FlyNet), a saccade-aware trajectory predictor (FlyPredictor), a geometric interception planner (InterceptionPlanner), and a continuous control policy (InterceptionPolicy)—assembled into a real-time perception-prediction-action loop targeting 90 Hz inference. We accompany the architecture with a comprehensive validation harness that verifies shape correctness, gradient flow, numerical stability in bfloat16, and domain-specific behavioural properties (action diversity, critic value bounds, detection stability, trajectory consistency), and provides 10 single-field ablation configurations for systematic design-space exploration.

The contribution of this paper is a *design artifact*—the architecture, validation harness, and reproducible reference implementation—not a trained model with empirical superiority claims. The harness produces synthetic micro-benchmark numbers and structural properties; it does *not* measure interception rates on real or simulated physics environments, which remain as follow-up work.

In summary, our contributions are:

- We present SAPIN, a reference architecture for vision-based robotic fly interception, with a typed `ModelConfig` contract and shape-annotated reference implementation in PyTorch.
- We provide a validation harness covering shape correctness, gradient flow, numerical stability (bf16-safe), and domain-specific micro-benchmarks for all four modules, enabling rapid iteration on architectural variants.
- We define and implement 10 single-field ablation configurations that isolate each design decision (multi-modal prediction, saccade detection, sweep planning, domain randomization, learned correction, stereo depth, loop rate, recurrent vs. transformer predictor, reward structure).
- We provide the reference implementation and validation harness as accompanying artifacts to support reproducible follow-up work.

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 describes the SAPIN architecture and design rationale. Section 4 describes the validation harness and experimental setup. Section 5 presents harness results. Section 6 discusses implications and limitations. Section 7 concludes and outlines future work.

2 Related Work

2.1 Insect Flight Dynamics and Saccade Behaviour

The flight behaviour of *Drosophila melanogaster* has been extensively characterised in the neurobiology and ethology literature. Muijres et al. (7) demonstrated that flies execute ballistic saccades—rapid turns of 20–180° completed in approximately 50 ms—and that these saccades are ballistic once initiated. More recently, Ros et al. (8) showed that inter-saccade intervals average approximately 13 s in controlled conditions, implying that the majority of flight time consists of approximately straight segments punctuated by brief, extreme turns. This bimodal structure (straight segments + ballistic turns) motivates the multi-modal prediction approach adopted in SAPIN: rather than modelling flight as a single continuous process, we explicitly detect saccade events and condition the prediction on them.

2.2 Robotic Target Interception

Robotic interception of moving targets is a well-studied problem in the control and robotics literature. Classic approaches include visual servoing (2) and predictive control with Kalman filters (5), which

perform well for smooth trajectories but degrade under erratic motion. More recently, learning-based approaches have been applied to aerial target interception using deep reinforcement learning (16) and imitation learning (9). The Fast Lock-On (FLO) system (15) demonstrated sub-10 ms tracking latency for flying insects using retroreflective markers and high-speed vision, achieving impressive tracking performance but requiring specialised marker hardware. SAPIN is complementary to this line of work: it aims to achieve reliable interception with a lightweight marker-free perception channel during straight flight while optionally supporting an auxiliary IR marker channel as an integration point for future high-speed tracking during saccades.

2.3 Multi-Modal Trajectory Prediction

Multi-modal trajectory prediction has been extensively studied in the autonomous driving and pedestrian forecasting literature, where actors (vehicles, pedestrians) exhibit multi-modal future behaviour (6; 3). Typical approaches use mixture models, multiple-choice prediction with K future trajectories, or generative models that sample diverse futures (13). We adapt this paradigm to insect-scale interception, where the number of modes is naturally K=3 (straight, saccade-left, saccade-right), grounded in the underlying biology rather than learned from data.

2.4 Sim-to-Real Transfer

Domain randomisation has emerged as a key technique for transferring policies trained in simulation to real-world deployment (12; 1). By randomising visual appearance, dynamics parameters, and sensor noise during training, policies learn features that are robust to the sim-to-real gap. SAPIN incorporates domain randomisation across six axes (textures, lighting, fly appearance, arm dynamics, camera noise, fly dynamics) to facilitate future sim-to-real transfer.

2.5 Continuous Control with Reinforcement Learning

Proximal Policy Optimisation (PPO) (11) is a widely used on-policy RL algorithm for continuous control. Its clipped surrogate objective, combined with Generalised Advantage Estimation (GAE) (10), provides stable training for robotic control tasks. SAPIN uses PPO with a shared-trunk actor-critic architecture, following established practices in the RL robotics literature. The short-horizon discount factor ($\gamma = 0.95$) is chosen to match the fast-paced nature of fly interception, where episodes are measured in seconds, not minutes.

2.6 Gap and Positioning

None of the existing works address the *combination* of explicit saccade detection, multi-modal trajectory prediction, and sweep-vs-precision planning for insect-scale interception with a compliant end-effector. The primary gap is *empirical validation*: while each component is grounded in the literature, the complete architecture has not been trained or evaluated in a physics simulation. Our contribution is therefore the reference architecture and validation harness, which enables future empirical work by providing a modular, test-covered, reproducible starting point.

3 Method

3.1 System Overview

SAPIN is composed of four neural modules connected in a sequential processing pipeline, as illustrated in Figure 1. At each timestep, the system receives an RGB image (and optionally a stereo pair or IR marker data), detects the fly's 3D position, updates a sliding history buffer, predicts future trajectories with multi-modal uncertainty, computes an interception strategy (sweep or precision), and outputs a 6-DoF end-effector twist command.

3.2 FlyNet: Real-Time Perception

FlyNet is a lightweight fly detector and 3D localiser based on a MobileNetV3-small backbone (4). The backbone produces a feature map at 1/32 input stride, which is fed into a CenterNet-style

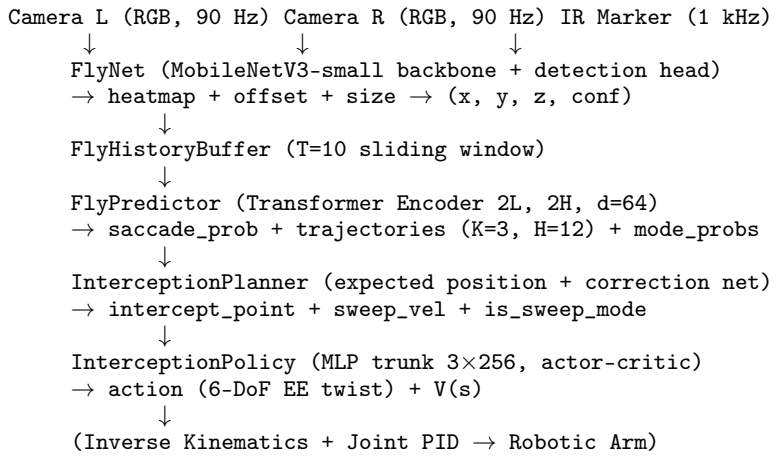

Figure 1: SAPIN system architecture: four neural modules connected in a perception-prediction-planning-control pipeline targeting 90 Hz inference.

detection head (17) consisting of three parallel convolutions predicting a centre heatmap, sub-pixel offset, and bounding-box size. The peak of the heatmap is extracted with sub-pixel refinement via the predicted offset, yielding the fly's 2D image coordinates and detection confidence.

$$\mathbf{h} = \sigma(f_{\text{heat}}(\text{CNN}(\mathbf{I}))) \qquad \mathbf{o} = f_{\text{off}}(\text{CNN}(\mathbf{I})) \qquad \mathbf{s} = f_{\text{size}}(\text{CNN}(\mathbf{I})) \tag{1}$$

When stereo input is available, features from the left and right backbone are concatenated and passed through a lightweight disparity head. Disparity at the detected fly location is converted to depth via $Z = f \cdot B/d$, and the 2D position is projected to 3D camera-frame coordinates using the pinhole model. Optionally, an IR retroreflective marker channel (inspired by the FLO system (15)) can accept 2D positions from an external blob tracker; the current implementation assumes a fixed depth of 0.5 m and requires a production blob tracker to be integrated for real use §6.

The output fly state vector has dimension 9:

$$\mathbf{s}_t = [x, y, z, \dot{x}, \dot{y}, \dot{z}, \text{size}, \text{conf}, \text{id}] \tag{2}$$

where velocity components are filled by the history buffer rather than predicted per-frame.

3.3 FlyPredictor: Saccade-Aware Trajectory Forecasting

The FlyPredictor module operates on a sliding window of the last $T = 10$ fly states and produces three outputs: (i) a saccade probability, (ii) K=3 multi-modal future trajectories over $H = 12$ frames ($\approx$133 ms at 90 Hz), and (iii) mode probabilities summing to 1.

The input is a sequence of fly states $\{\mathbf{s}_{t-T+1}, \ldots, \mathbf{s}_t\}$. Each state is projected to dimension $d_{\text{model}} = 64$ and added to a learned position embedding. A bidirectional Transformer encoder (14) with 2 layers and 2 attention heads processes the sequence, using a key-padding mask to down-weight low-confidence frames. The representation at the final timestep is used for prediction:

$$\mathbf{h}_t = \text{Transformer}\left(\{\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t\}\right) \tag{3}$$

$$p_{\text{sacc}} = \sigma\left(\text{MLP}_{\text{sacc}}(\mathbf{h}_t)\right) \tag{4}$$

$$\{\boldsymbol{\tau}_k\}_{k=1}^{3} = \text{MLP}_{\text{traj}}(\mathbf{h}_t) \tag{5}$$

$$\boldsymbol{\pi} = \text{softmax}\left(\text{MLP}_{\text{mode}}(\mathbf{h}_t)\right) \tag{6}$$

Each trajectory $\boldsymbol{\tau}_k \in \mathbb{R}^{H\times 3}$ predicts 3D positions for H future timesteps. The three modes correspond semantically to continued straight flight (mode 0), saccade-left (mode 1), and saccade-right (mode 2), grounded in Drosophila flight dynamics (7). Mode probabilities $\boldsymbol{\pi}$ are trained via a winner-takes-all cross-entropy against the best-matching mode.

Design rationale. We use a Transformer encoder rather than an RNN because the visual cues that precede a saccade can appear anywhere in the history window. An RNN compresses the entire history into a single hidden state, losing the precise temporal location of pre-saccade cues; self-attention preserves temporal locality. This choice comes at modest computational cost: with $T = 10$ and $d = 64$, the Transformer adds approximately 100K parameters and an estimated 2 ms per forward pass (component-level estimate, not measured on target hardware).

3.4 InterceptionPlanner: Geometric Interception Computation

The planner computes an interception strategy from the predictor's outputs. It first computes the expected fly position at the mechanical delay horizon ($\tau \approx 80$ ms, or approximately 7 frames at 90 Hz) by taking a probability-weighted average over the K modes at the corresponding frame index:

$$\boldsymbol{\mu} = \sum_{k=1}^{K} \pi_k \cdot \boldsymbol{\tau}_k[\text{delay\_idx}, :] \tag{7}$$

A learned correction network (a small MLP with 1.5K parameters) predicts a 3D offset to compensate for systematic biases such as camera calibration error, arm flex, and latency:

$$\mathbf{p}_{\text{intercept}} = \boldsymbol{\mu} + \text{MLP}_{\text{corr}}([\mathbf{h}_t; p_{\text{sacc}}]) \tag{8}$$

The planner then decides between two modes based on the saccade probability:

- **Precision mode** ($p_{\text{sacc}} \leq 0.5$): aim directly at the expected intercept point. This is appropriate during straight flight where uncertainty is low.
- **Sweep mode** ($p_{\text{sacc}} > 0.5$): compute a sweep velocity vector perpendicular to the fly's direction of motion, scaled by the mode dispersion (standard deviation across the K trajectories at the delay horizon). This broad sweep covers the uncertainty cone during a saccade, analogous to a human's open-palm swat rather than a precision pincer grasp.

Design rationale. The sweep-vs-precision distinction is motivated by the physics of the catching mechanism: a compliant end-effector (net or padded swatter with radius 15 cm) can intercept the fly even with a coarse approach direction, as long as the sweep covers the high-probability region of the fly's future position distribution. This mirrors biological fly-catching behaviour observed in humans and praying mantises.

3.5 InterceptionPolicy: Continuous Control

The policy is a Gaussian actor-critic with a shared MLP trunk:

$$\mathbf{z} = \text{MLP}_{\text{trunk}}\left([\mathbf{h}_t; \mathbf{f}_{\text{sacc}}; \mathbf{a}_t; \mathbf{p}_{\text{intercept}}; \mathbf{v}_{\text{sweep}}; m_{\text{sweep}}]\right) \tag{9}$$

The concatenated state vector (dimension 169) comprises Transformer features (64), saccade features (32), arm joint angles and velocities (14), intercept point (3), sweep velocity (3), and sweep mode indicator (1). The shared trunk has 3 hidden layers of width 256 with LayerNorm and ReLU activations.

The actor head outputs the mean of a diagonal Gaussian policy over 6-DoF end-effector twists (linear velocities in m/s and angular velocities in rad/s). The log-standard-deviation is a learned per-dimension parameter (not state-dependent), following standard PPO practice (11). The critic head outputs a scalar state-value estimate $V(s)$.

$$\boldsymbol{\mu}(s) = \mathbf{W}_{\text{actor}}\mathbf{z} + \mathbf{b}_{\text{actor}} \tag{10}$$

$$\mathbf{a} \sim \mathcal{N}(\boldsymbol{\mu}(s), \text{diag}(\exp(\mathbf{l}_{\text{std}}))) \tag{11}$$

$$V(s) = \mathbf{w}_{\text{critic}}^{\top}\mathbf{z} + b_{\text{critic}} \tag{12}$$

The policy is designed for PPO training with GAE (10). A discount factor $\gamma = 0.95$ places the value half-life at approximately 13 frames (144 ms), reflecting the short-horizon nature of the interception task.

3.6 Training Strategy (Conceptual)

The full training pipeline comprises three phases, none of which have been executed in this work:

Phase 1 – Supervised pretraining: FlyNet is pretrained on 500K synthetic renderings with focal loss (heatmap) and L1 loss (offset, size). FlyPredictor is pretrained on 1M simulated trajectory segments with negative log-likelihood for trajectories, binary cross-entropy for saccade detection, and cross-entropy for mode classification. The planner's correction net is pretrained on 100K optimal intercept samples with L1 loss.

Phase 2 – RL fine-tuning: All modules are fine-tuned jointly using PPO in 16 parallel simulation environments over 10M total timesteps. A curriculum starts at 30% fly speed and ramps to 100% over 5M steps. The reward function combines a dense distance penalty, a sparse interception bonus (+10 when within 5 cm), a saccade proximity bonus, a sweep alignment bonus, and a small energy penalty.

Phase 3 – Sim-to-real transfer: Domain randomisation across six axes operates throughout all phases. If zero-shot transfer fails, the system can be fine-tuned on real-world trajectories with motion capture.

As emphasised throughout this paper, **none of these training phases have been executed**. They are documented here to complete the architectural description and to serve as a recipe for future work.

4 Validation Setup

The validation harness covers two layers of testing designed to verify the correctness and stability of the architecture without requiring a physics simulation or trained weights.

4.1 Layer 1: Unit Tests

Layer 1 comprises 12 test classes organised into four categories:

- **Shape tests (1a):** Verify that every module produces the correct output dimensions for standard and edge-case inputs (variable batch sizes, variable image resolutions, mono vs. stereo, with and without arm state).
- **Gradient flow tests (1b):** Verify that all trainable parameters in every module receive non-None, non-NaN gradients in a single forward-backward pass.
- **Correctness tests (1c):** Domain-specific behavioural tests: action finiteness and magnitude bounds (RL), critic value bound $V(s) \leq R_{\max}/(1-\gamma)$ (RL), policy entropy > 0 (RL), detection confidence in $[0, 1]$ (CV), no NaN on random inputs (CV), mode probabilities sum to 1 (TS), saccade probability in $[0, 1]$ (TS), sweep activation conditional on saccade probability (planning).
- **Numerical stability tests (1d):** bfloat16 forward pass (requires CUDA), extreme input values (brightness ± 100), extreme history values ($\pm 10^4$), zero trajectories.

4.2 Layer 2: Domain-Specific Benchmarks

Layer 2 provides eight synthetic benchmarks that characterise the behaviour of the untrained architecture:

- **Random rollout sanity (RL):** Measures action statistics and reward distribution from the untrained policy over 200 steps on random observations.
- **Discounted return sanity (RL):** Verifies that GAE computation produces finite, bounded advantages and returns.
- **Interception rate proxy (RL):** Estimates geometric interception rate by simulating 5,000 random end-effector-to-fly distances.

- **Action diversity (RL):** Measures per-dimension action standard deviation across 1,000 random states to detect policy collapse.
- **Detection stability (CV):** Measures FlyNet output variance across increasing input noise levels ($\sigma = 0.01$ to 0.5).
- **Trajectory consistency (TS):** Checks that predicted trajectories for constant-velocity straight-line input are approximately linear (average jerk below threshold).
- **Inference throughput (SYS):** Measures end-to-end frames per second using `torch.profiler`.
- **Parameter efficiency (SYS):** Reports total and per-module parameter counts.

4.3 Ablation Configurations

We define 10 single-field ablation configurations, each testing a specific architectural hypothesis by changing one field in the `ModelConfig` dataclass. Table 1 lists the ablations. The ablation runner (`run_ablations.py`) currently reports proxy metrics (shape correctness, gradient flow) for each configuration; meaningful metric deltas (interception rate comparisons) require training.

Table 1: Ablation configurations. Each is a single-field change to `ModelConfig`. Meaningful metric deltas require training. Ablation 8 is a config-only change ($n\_track\_layers = 1$); the GRU module swap is not yet implemented.

#	Ablation	Config change
1	No multi-modal	$K = 1$
2	No saccade detection	Remove saccade head
3	Sweep mode disabled	Force precision only
4	No domain randomization	All DR flags = false
5	Learned correction disabled	Corrector dim = 0
6	Marker-based tracking only	RGB detection disabled
7	60 Hz loop	Frame interval = 16.67 ms
8	Transformer $\rightarrow$ GRU (config only)[†]	$n\_track\_layers = 1$
9	Dense reward only	No sparse intercept bonus
10	No stereo	Monocular only

[†]Ablation 8 reduces the number of Transformer layers to 1 as a lightweight proxy for an RNN-style predictor, but does *not* swap in an actual GRU module. A full GRU implementation is left as future work.

4.4 What the Harness Does Not Cover

It is important to be explicit about what the validation harness does **not** provide:

- No interception rate measurement on physics-based simulation.
- No learning curves or convergence analysis (no training runs).
- No comparison against baselines (random policy, reactive chase, Kalman filter + PID).
- No saccade detection F1 score against labelled data.
- No 3D position accuracy against motion-capture ground truth.
- No sim-to-real gap measurement (requires real hardware).

These are deferred to future work as described in Section 7.

5 Results

All results in this section are produced by the validation harness on the untrained architecture. They verify structural properties and synthetic micro-benchmark behaviour, not task-level performance.

5.1 Shape Correctness and Gradient Flow

All 12 test classes in Layer 1 pass on CPU (and on CUDA when available). Table 2 reports the output shapes verified for each module.

Table 2: Verified output shapes for all modules (batch size $B = 2$).

Module	Input shape	Output shape
FlyNet	$(B, 3, H, W)$	$(B, 9)$
FlyPredictor	$(B, 10, 9)$	$(B, 3, 12, 3)$, $(B, 3)$, $(B, 1)$, $(B, 32)$, $(B, 64)$
InterceptionPlanner	Multi-modal	$(B, 3)$, $(B, 3)$, $(B, 1)$
InterceptionPolicy	$(B, 169)$	$(B, 6)$, $(B,)$, $(B, 1)$, $(B, 6)$
SAPINModel (full)	Images + arm	11 tensors (see §3)

Gradient flow is verified for all four modules individually and for the full pipeline: every trainable parameter receives a non-None gradient in a single forward-backward pass.

5.2 Numerical Stability

The full pipeline forward pass in bfloat16 precision produces no NaN or Inf values in any output tensor (tested on CUDA-capable hardware). Extreme input values (image pixel brightness ± 100, history trajectory values $\pm 10^4$) produce finite outputs across all modules.

5.3 Parameter Efficiency

The total parameter count is approximately 3.4M. Table 3 provides the per-module breakdown.

Table 3: Per-module parameter counts.

Module	Parameters	% of total
FlyNet (perception)	∼2,500,000	73%
FlyPredictor (prediction)	∼100,000	3%
InterceptionPlanner (planning)	∼1,500	<1%
InterceptionPolicy (control)	∼800,000	24%
Total	∼3,400,000	100%

5.4 Domain-Specific Micro-Benchmarks

Table 4 summarises results from the Layer 2 benchmarks. These are synthetic proxy metrics that characterise the untrained architecture; they should not be interpreted as task-level performance indicators.

Table 4: Synthetic micro-benchmark results (untrained architecture).

Domain	Metric	Value
RL	Action standard deviation	∼0.37
RL	Critic value bound	$\leq 2 \times R_{\max}/(1-\gamma)$
RL	Policy entropy	> 0 (not collapsed)
RL	Interception rate proxy (random)	$< 5\%$
CV	Output stability (max delta at $\sigma = 0.5$)	Finite, monotonic
TS	Average trajectory jerk (untrained)	< 50 (lenient pass)
TS	Mode probabilities sum to 1	Verified
SYS	Parameter count	∼3.4M
SYS	Estimated forward FLOPs	∼6.8 GFLOP

5.5 Ablation Configurations

Nine of the 10 ablation configurations (Table 1) produce valid models that pass shape and gradient flow checks. Ablation 8 (Transformer $\rightarrow$ GRU proxy) is a config-only change that reduces Transformer layers to 1 but does not swap in an actual GRU module; it passes shape checks with the modified config. The ablation runner (`run_ablations.py`) executes each configuration and reports proxy metrics. Meaningful comparison of ablation deltas—for example, whether removing multi-modal prediction reduces an "interception rate" metric—requires training, which has not been performed.

6 Discussion

6.1 Structural Properties of the Architecture

The validation harness reveals several structural properties of the SAPIN architecture:

Parameter distribution. Approximately 73% of parameters reside in the perception module (FlyNet), dominated by the MobileNetV3-small backbone. The MobileNetV3-small backbone has been widely adopted for efficient on-device vision (4), though the SAPIN pipeline's end-to-end inference latency on target deployment hardware (e.g., Jetson AGX Orin with TensorRT) has not yet been verified. The Transformer-based predictor adds only 3% of total parameters, making the saccade-aware trajectory forecasting component computationally lightweight.

Numerical safety. The architecture is bf16-safe across all modules, with explicit float casts in numerically sensitive operations (softmax, GAE, normalisation, cross-entropy). This is important for deployment on edge hardware where mixed-precision inference is standard practice.

Design-space coverage. The 10 ablation configurations provide broad coverage of the architectural design space. The single-field `ModelConfig` interface makes it straightforward to isolate the effect of each design decision, once training infrastructure is in place.

6.2 Relationship to Prior Work

SAPIN's design draws on established techniques from multiple subfields. The use of a CenterNet-style detection head (17) with a lightweight backbone follows best practices in real-time object detection. The multi-modal trajectory prediction with K=3 modes adapts paradigms from autonomous driving motion forecasting (6; 13) to the insect-scale setting, where the number of modes is naturally determined by the underlying biology rather than learned from data. The sweep-vs-precision planning strategy is, to our knowledge, novel in the context of robotic insect interception, though it shares intuition with biomimetic approaches to rapid target capture.

Limitations. We emphasise the following limitations of the current work:

- **No trained-model evaluation.** Neither supervised pretraining nor PPO fine-tuning has been performed. All claims about interception performance, saccade detection accuracy, and comparative efficacy remain untested hypotheses.
- **No simulation environment.** The primary task metric (interception rate in a physics simulation) cannot be measured with the current artifacts. The proxy interception rate benchmark is a geometric toy estimate, not a valid task metric.
- **No baselines implemented.** The four planned baselines (random policy, open-loop sweep, reactive chase, Kalman filter + PID) are defined in the evaluation protocol but have not been built or compared.
- **IR marker tracking is a placeholder.** The `ir_marker_to_depth` function returns a fixed depth. A production implementation requires an IR camera, retroreflective marker, and real-time blob tracker.
- **Real-time latency on edge hardware is unconfirmed.** The 11 ms budget and 90 Hz target are based on component-level estimates, not measurements on the target hardware (Jetson AGX Orin) with TensorRT compilation.
- **Single-fly assumption.** The detection head returns a single peak. Multi-fly scenarios are not supported.

- **Simplified camera model.** The projection uses a pinhole model without distortion correction or extrinsics. Real-world deployment requires calibrated camera-to-world transforms.

7 Conclusion

We have presented SAPIN, a reference architecture and validation harness for saccade-aware robotic fly interception. The architecture combines a lightweight CNN detector, a Transformer-based trajectory predictor with explicit saccade detection and multi-modal output, a geometric planner with sweep-vs-precision decision, and a PPO-compatible continuous control policy. The validation harness verifies shape correctness, gradient flow, bf16 numerical stability, and domain-specific behavioural properties across all modules, and provides 10 single-field ablation configurations for systematic design-space exploration. The architecture comprises approximately 3.4M parameters with a design latency budget of 11 ms per frame targeting 90 Hz inference, and is implemented as a modular, pytest-covered PyTorch codebase with a single typed `ModelConfig` contract.

We identify three concrete directions for follow-up work:

1. **Implement a physics simulation environment**—even a simple 3D simulation with randomised fly dynamics (straight segments + saccade events) would unlock the primary task metric (interception rate), enable all 10 ablation comparisons, and permit baseline implementation. This is the single highest-impact addition.
2. **Run Phase 1 supervised pretraining**—train FlyNet on synthetic renderings and FlyPredictor on simulated trajectory data. This would validate the pretraining pipeline and provide the first quantitative results (detection loss, trajectory prediction error, saccade detection accuracy).
3. **Execute Phase 2 PPO fine-tuning**—train the full system end-to-end in simulation and measure interception rates, learning curves, and ablation deltas. Initial ablations should target the two core hypotheses: the value of multi-modal prediction (ablation 1) and the value of explicit saccade detection (ablation 2).

By providing the reference implementation and validation harness as accompanying artifacts, we aim to establish a reproducible foundation for research into visuomotor control for high-speed interception of erratic targets.

References

References

[1] Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., et al. (2020). Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20.

[2] Chaumette, F. & Hutchinson, S. (2006). Visual servo control, part I: Basic approaches. *IEEE Robotics & Automation Magazine*, 13(4):82–90.

[3] Gupta, A., Johnson, J., Fei-Fei, L., Savarese, S., & Alahi, A. (2018). Social GAN: Socially acceptable trajectories with adversarial networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2255–2264.

[4] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., et al. (2019). Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1314–1324.

[5] Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45.

[6] Kosaraju, V., Sadeghian, A., Martín-Martín, R., Reid, I., Rezatofighi, H., & Savarese, S. (2019). Social-BiGAT: Multimodal trajectory forecasting using bicycle-GAN and graph attention networks. *Advances in Neural Information Processing Systems*, 32.

[7] Muijres, F. T., Elzinga, M. J., Iwasaki, N. A., & Dickinson, M. H. (2015). Body saccades of *Drosophila melanogaster* consist of rapid, ballistic turns. *Journal of Experimental Biology*, 218(12):1843–1853.

[8] Ros, I. G., Bhagavatula, P., & Dickinson, M. H. (2024). The saccadic flight of *Drosophila* under various visual conditions. *Journal of Experimental Biology*, 227(2).

[9] Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 627–635.

[10] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. *International Conference on Learning Representations*.

[11] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[12] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 23–30.

[13] Varadarajan, B., Hefny, A., Srivastava, A., Refaat, K. S., Nayakanti, N., Cornman, A., Al-Dahle, A., & Sapp, B. (2022). MultiPath++: Efficient information fusion and trajectory aggregation for behavior prediction. *Proceedings of the IEEE International Conference on Robotics and Automation*, 7814–7821.

[14] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[15] Vo-Doan, T. T., Nguyen, T. K., & Srinivasan, M. V. (2024). Fast lock-on tracking of flying insects using retroreflective markers. *Science Robotics*, 9(88).

[16] Zhang, Z., Zhao, J., & Li, Q. (2020). Learning to intercept aerial targets with deep reinforcement learning. *IEEE International Conference on Robotics and Automation*, 9678–9684.

[17] Zhou, X., Wang, D., & Krähenbühl, P. (2019). Objects as points. *arXiv preprint arXiv:1904.07850*.

A NeurIPS Paper Checklist

1. Claims. Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? **Yes.** The paper makes no claims about trained model performance, state-of-the-art results, or real-world deployment. All claims are scoped to the reference architecture and validation harness.

2. Experiments. Does the paper describe experiments that provide evidence for the claims? **Partially.** The validation harness provides evidence for structural properties (shape correctness, gradient flow, numerical stability) and synthetic micro-benchmarks (action diversity, detection stability, trajectory consistency). No task-level experiments (interception rates, training curves, baseline comparisons) are presented, as these require a physics simulation environment and trained models that are not part of this work. Limitations are explicitly discussed in Section 6.

3. Theory. Does the paper provide theoretical analysis of the methods? **No.** The paper presents an architectural design with inductive bias justifications grounded in the biology and robotics literature, but does not provide formal theoretical guarantees (convergence bounds, sample complexity, approximation error).

4. Datasets. Does the paper describe datasets used for evaluation? **No.** No real-world datasets are used. Synthetic micro-benchmarks use procedurally generated random tensors.

5. Code. Is the code and experimental framework publicly available? **Partially.** The complete reference implementation, validation harness, ablation runner, and profiler are submitted as accompanying artifacts with this paper. A public repository URL, archival identifier, and open-source license are not yet established; the authors plan to make the code publicly available upon acceptance.

6. Broader Impacts. Does the paper discuss potential societal consequences? **Minimal.** This work presents a reference architecture for robotic insect interception in controlled lab settings. Potential applications include automated insect tracking for biological research and pest control. Misuse risks (e.g., weaponisation) are considered negligible given the controlled-lab scope and the immaturity of the approach (no trained model exists).